



廣東工業大學

GUANGDONG UNIVERSITY OF TECHNOLOGY

嵌入式系统课程设计报告

《基于 STM32F103C8T6 的智能手表设计》

姓 名： 柳政道

学 号： 3124009572

专 业： 微电子科学与工程

班 级： 微电子 1 班

队 名： mygo

队 友： 刘缘圆、王昊

指导老师： 周贤中



集成电路学院

2026 年 7 月 12 日

摘要

本项目基于 STM32F103C8T6 微控制器，采用 FreeRTOS 多任务实时调度架构，设计并实现了一款多功能智能手表原型系统。硬件层面集成了 0.96 寸 SSD1306 OLED 显示屏、MPU6050 六轴陀螺仪、HC-05 蓝牙串口模块、EC11 旋转编码器及锂电池独立供电链路（TP4056 + AMS1117）。软件层面基于 FreeRTOS 实现了 5 个任务的协同调度，包括时钟维护、传感器数据采集、OLED 渲染、编码器菜单导航和蓝牙数据发送。在课程设计标准要求基础上，额外实现了秒表与倒计时工具、手电筒模式、5 档亮度调节（进度条动画）、蓝牙开关控制、中英文多字体显示、自动休眠与唤醒、卡路里/距离健康估算、Flask Web 上位机及锂电池独立供电等创新功能。系统经长时间运行验证，FreeRTOS 调度稳定，无死机、无堆栈溢出。

关键词：STM32；FreeRTOS；智能手表；MPU6050；SSD1306；蓝牙通信；旋转编码器

目录

1	项目简介	1
1.1	项目背景	1
1.2	主要功能概述	1
2	项目设计目标	2
3	项目设计与实现	2
3.1	硬件设计	2
3.1.1	系统总体架构	2
3.1.2	主控制器模块	3
3.1.3	OLED 显示模块	3
3.1.4	MPU6050 六轴传感器模块	3
3.1.5	蓝牙通信模块	4
3.1.6	旋转编码器输入模块	4
3.1.7	电源模块	4
3.1.8	PCB 设计	4
3.1.9	电路接线	5
3.2	软件设计	6
3.2.1	系统软件架构	6
3.2.2	主程序流程	7
3.2.3	关键技术实现	7
4	功能展示	9
4.1	功能描述	9
4.2	演示结果	9
4.2.1	主界面与菜单系统	9
4.2.2	核心功能页面	10
4.2.3	时钟工具	10
4.2.4	手电筒功能	11
4.2.5	系统设置	11
4.2.6	蓝牙传输与 PC 上位机	12

4.2.7	自动休眠	12
4.2.8	关于页面——多字体中英文显示	13
5	物料与成本	13
5.1	物料清单	13
5.2	项目总成本	13
5.3	损耗与损坏记录	14
6	个人贡献	14
6.1	编码工作	14
6.2	硬件连接与调试	14
6.3	报告撰写	14
6.4	资本投入	15
6.5	开发问题描述与解决方案	15
7	项目成果与反思	15
7.1	成果展示	15
7.2	项目评估	16
7.2.1	成功之处	16
7.2.2	不足之处	16
7.3	个人反思	17
8	附录	17
8.1	设计图纸	17
8.2	完整代码清单	18
8.3	参考文献	18

1 项目简介

1.1 项目背景

智能手表作为可穿戴设备的典型代表，集成了嵌入式微控制器、MEMS 传感器、无线通信、实时操作系统及人机交互等多项技术。随着物联网和智能硬件的快速发展，可穿戴设备在健康监测、运动追踪和个人信息管理等领域展现出巨大的应用潜力。

本课程设计以 STM32F103C8T6 微控制器为核心平台，采用渐进式开发策略，从裸机程序逐步迁移至 FreeRTOS 多任务实时调度架构，依次集成 OLED 显示屏、MPU6050 六轴陀螺仪、HC-05 蓝牙串口模块及 EC11 旋转编码器，最终实现了一款功能完备的智能手表原型系统。

1.2 主要功能概述

本智能手表具备以下核心功能：

1. **时间日期与步数显示（主界面）**：实时显示当前时间、日期、累计步数，支持通过蓝牙发送时间同步帧校准日期时间
2. **多级菜单系统**：旋转编码器按压进入 MENU 主菜单，旋转翻页，短按进入子菜单，长按返回上级。10 秒无操作自动返回主界面，主界面 5 秒无操作自动熄屏
3. **STEPS 页面**：基于 MPU6050 加速度计的峰值检测 + 动态阈值算法，实时统计步数，并估算卡路里消耗和运动距离
4. **SENSOR 页面**：显示三轴加速度（X/Y/Z）、俯仰角（Pitch）、温度、电池电压，数据以 200 Hz 频率采集
5. **CLOCK 页面**：提供秒表（开始/暂停/复位）和倒计时器（编码器设置时间/开始/暂停/复位）
6. **FLASHLIGHT 页面**：OLED 屏幕全白最大亮度，作为应急照明使用
7. **SETTING 页面**：5 档亮度调节（可视化进度条动画）、蓝牙开关控制、关于页面（项目名称与小组成员，中英文多字体显示）
8. **蓝牙无线传输**：HC-05 以 1 Hz 频率持续发送文本格式的传感器数据包（加速度、俯仰角、温度、步数、卡路里），手机端任意蓝牙串口 APP 即可查看

9. **PC 上位机**：基于 Python Flask 的 Web 端实时传感器数据可视化面板，浏览器即开即用
10. **锂电池供电**：TP4056 充电管理 + AMS1117 稳压 + 3.7V 锂电池，实现可充电独立供电，无需外接电源即可运行

2 项目设计目标

本项目在设计初期制定了以下目标和预期成果：

表 1 设计目标与预期成果

目标	预期成果
多任务实时调度	基于 FreeRTOS 实现 5 个独立任务的稳定调度
传感器数据采集	MPU6050 以 200 Hz 采样率稳定读取加速度和角速度，姿态角误差 $< 2^\circ$
计步算法	正常行走场景下计步准确率 $> 85\%$
蓝牙数据链路	1 Hz 频率稳定传输传感器数据包，手机端蓝牙串口 APP 可实时查看
菜单交互体验	旋转编码器流畅切换页面，按键防抖，无明显响应延迟
低功耗设计	自动熄屏、自动回主页、锂电池独立供电，系统正常运行功耗 $< 50\text{ mA}$
系统稳定性	可长时间连续运行无死机、无内存泄漏、无堆栈溢出
代码质量	模块化驱动设计，每种外设独立.c/.h 文件，关键代码含注释

3 项目设计与实现

3.1 硬件设计

3.1.1 系统总体架构

本智能手表的硬件系统以 STM32F103C8T6 最小系统板为核心，各外设模块通过 I2C、USART 和 GPIO 接口与 MCU 连接，锂电池经 AMS1117-3.3V 稳压后为整个系统供电。系统硬件架构如图 1 所示。

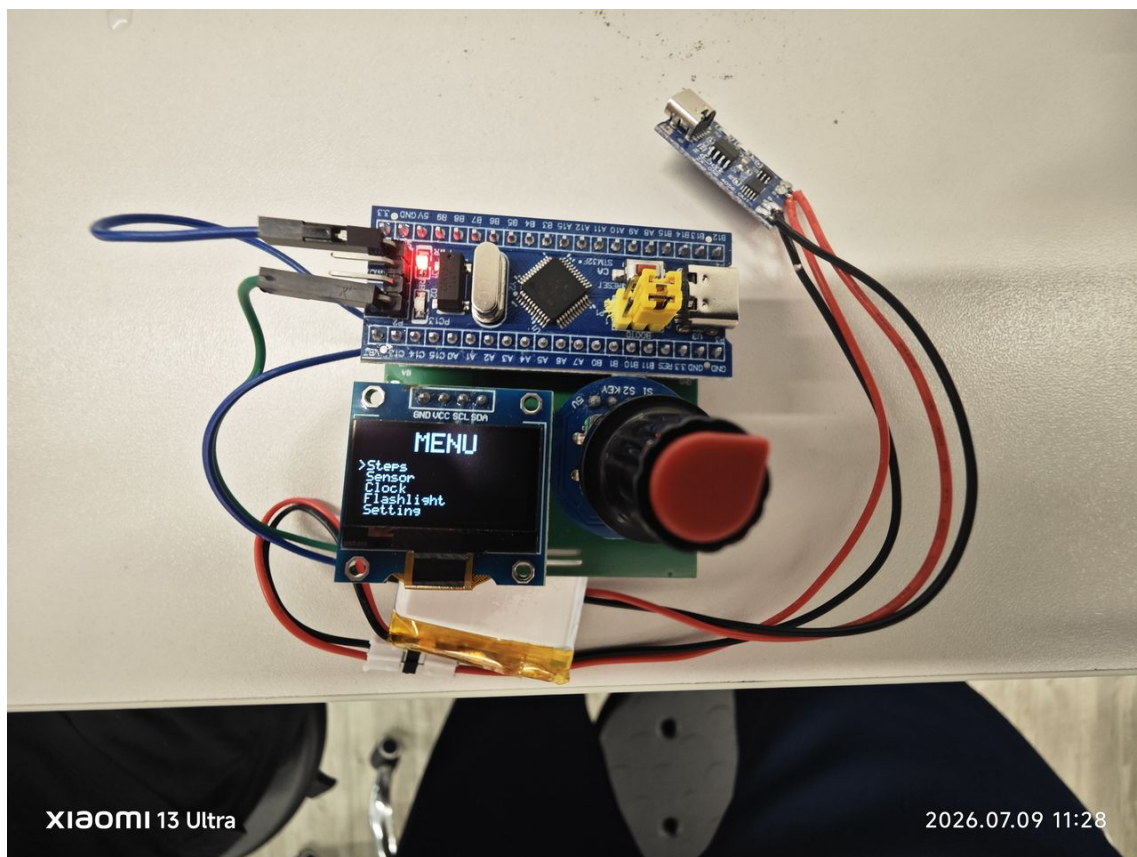


图 1 系统硬件连接实物图（OLED+ 编码器一体化模块与 MPU6050 共用 I2C1 总线）

3.1.2 主控制器模块

选用 STM32F103C8T6 作为主控芯片，该芯片基于 ARM Cortex-M3 内核，最高主频 72 MHz，内置 64 KB Flash 和 20 KB SRAM，集成 I2C、USART、USB、TIM 等丰富外设，能够满足智能手表对计算能力和外设接口的需求。

3.1.3 OLED 显示模块

采用 0.96 寸 MK993 OLED 模块，分辨率为 128×64 像素，控制器为 SSD1306（实际芯片为 SH1106 兼容），通过 I2C1 接口（PB6:SCL / PB7:SDA）与 MCU 通信，地址为 0x3C。该模块为 4 管脚版本（VCC / GND / SCL / SDA），不需外接复位引脚。

3.1.4 MPU6050 六轴传感器模块

采用 GY-521 模块，集成 MPU6050 芯片，包含 3 轴加速度计（±2g 量程）和 3 轴陀螺仪（±250°/s 量程），通过 I2C1 总线与 OLED 共享（设备地址 0x68）。以 200 Hz 采样率读取原始数据，在 MCU 端完成姿态解算。

3.1.5 蓝牙通信模块

采用 HC-05 蓝牙串口模块，通过 USART2 接口（PA2:TX / PA3:RX）与 MCU 通信，工作在从机透传模式。HC-05 将 MCU 发出的文本帧数据通过经典蓝牙 2.0 SPP 协议无线传输至手机端。

3.1.6 旋转编码器输入模块

采用 EC11 旋转编码器（与 OLED 一体化模块），A 相和 B 相分别连接 PA6、PA7（TIM3 编码器模式），按键连接 PB10（外部中断 EXTI10，下降沿触发）。旋转方向检测通过 TIM3 的编码器接口实现，按键通过中断 + 状态机去抖处理。

3.1.7 电源模块

采用 TP4056 充电管理模块 + AMS1117-3.3V 低压差 LDO 稳压模块 + 3.7V 2000mAh 聚合物锂电池的组合方案。TP4056 通过 Type-C 接口接入 5V 充电电源，同时对锂电池进行恒流/恒压充电。AMS1117 将电池电压（3.7-4.2V）稳压至 3.3V 为系统供电。该方案实现了可充电的便携供电功能，可脱离 USB 线独立运行。

3.1.8 PCB 设计

本项目的最终硬件形态由组员刘缘圆使用立创 EDA 完成 PCB 设计，将全部模块集成到一块电路板上，替代了前期面包板的临时连线方案。PCB 版图如图 2 所示。

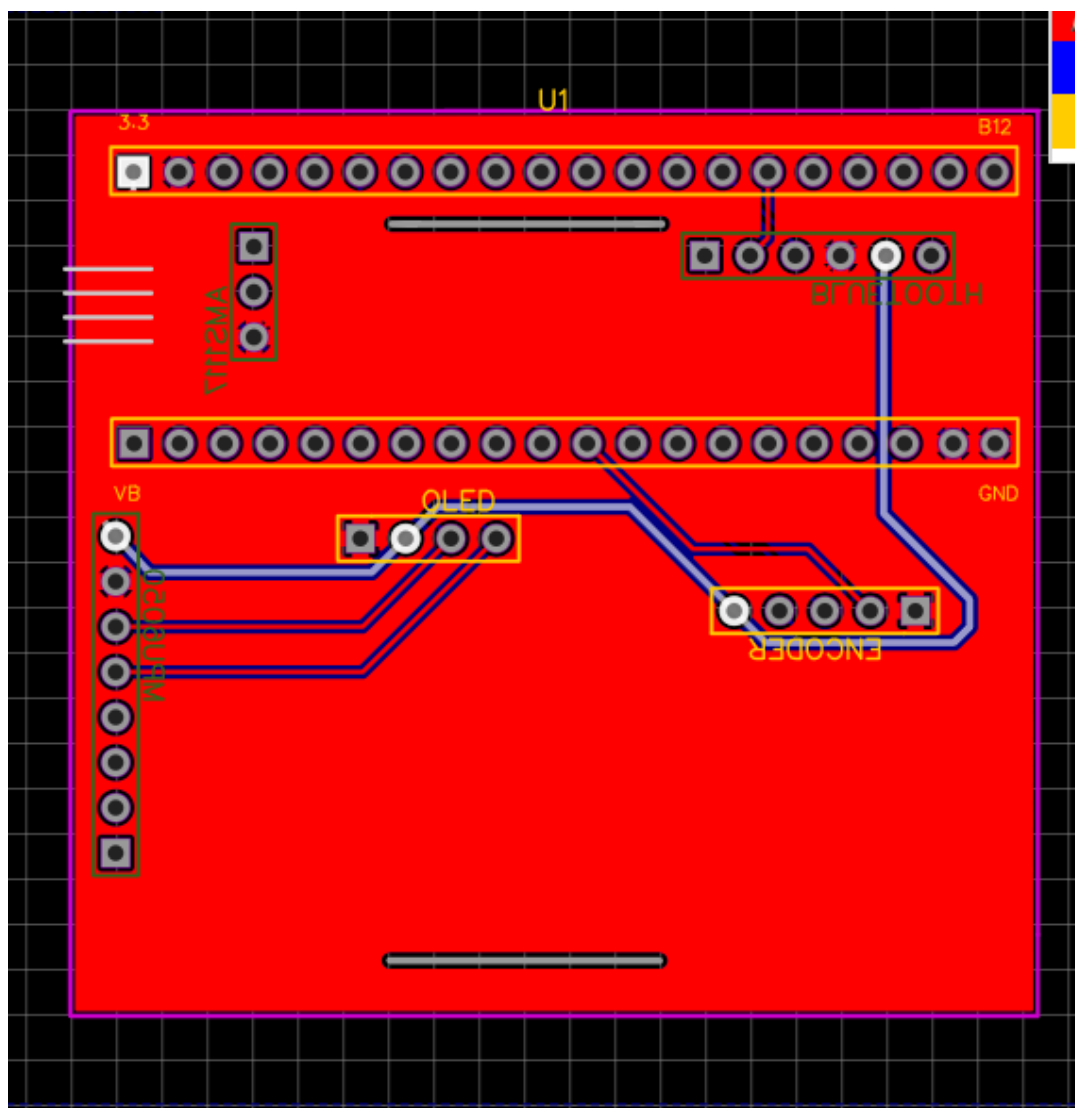


图 2 PCB 版图（刘缘圆设计）

3.1.9 电路接线

本项目的硬件接线方案由小组共同讨论确定。全系统各模块与 STM32 的引脚连接关系如图 3 和表 2 所示。

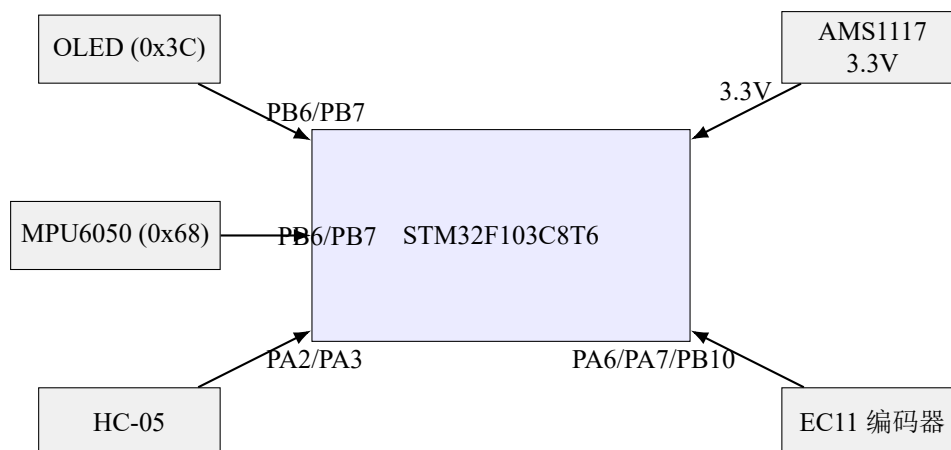


图 3 系统电路接线拓扑图

各模块与 STM32 的具体引脚连接关系见表 2。

表 2 全系统引脚接线表

STM32 引脚	连接模块	信号	说明
PB6	OLED + MPU6050	I2C1_SCL	两设备共享 I2C1 时钟线，4.7k Ω 上拉
PB7	OLED + MPU6050	I2C1_SDA	两设备共享 I2C1 数据线，4.7k Ω 上拉
PA2	HC-05	USART2_TX	STM32 发送 $\rightarrow$ 蓝牙 RXD，交叉连接
PA3	HC-05	USART2_RX	蓝牙 TXD $\rightarrow$ STM32 接收，交叉连接
PA6	EC11 编码器	TIM3_CH1	编码器 A 相，TIM3 编码器接口模式
PA7	EC11 编码器	TIM3_CH2	编码器 B 相
PB10	EC11 按键	EXTI10	下降沿中断 + 内部上拉
PB0	HC-05	STATE	蓝牙连接状态指示（高 = 已连接）
PA13	ST-Link	SWDIO	调试/烧录接口
PA14	ST-Link	SWCLK	调试/烧录接口
3.3V	所有模块	VCC	AMS1117 稳压输出供电
GND	所有模块	GND	公共地

3.2 软件设计

3.2.1 系统软件架构

本系统基于 FreeRTOS 实时操作系统，共设计了 5 个任务，按优先级从高到低如下：

表 3 FreeRTOS 任务配置

任务名	优先级	栈大小 (字)	核心职责
vTask_TimeKeep	High (4)	384	TIM2 秒中断驱动时钟维护, 管理系统空闲计
vTask_Sensor	AboveNormal (3)	512	MPU6050 200Hz 数据采集、姿态解算、计步
vTask_Display	Normal (2)	512	OLED 画面渲染、页面切换、自动休眠逻
vTask_Menu	Normal (1)	256	EC11 编码器 20ms 轮询、按键状态机、菜单
vTask_Bluetooth	BelowNormal (0)	384	1Hz 传感器数据文本帧发送、USART2 接收处理

任务间数据通信方案: Sensor 与 Display 之间通过全局结构体 `g_sensor` + 互斥锁 `mutexI2C` 进行线程安全的数据交换。所有 I2C 操作在各外设驱动函数内部自行加锁/解锁, 任务层不重复加锁, 避免了此前遇到的同任务双重加锁死锁问题。

3.2.2 主程序流程

1. 系统上电 → `HAL_Init()` 配置 NVIC 优先级分组和 TIM1 时基
2. `SystemClock_Config()` 配置 72 MHz 主频
3. 外设初始化: GPIO、USART2、I2C1、USB Device
4. `MX_FREERTOS_Init()`: 手动配置 SysTick, 创建 5 个任务、互斥锁、信号量
5. `osKernelStart()` 启动 FreeRTOS 调度器, 由操作系统接管所有任务
6. `vTask_TimeKeep` 最先运行: 初始化 OLED 显示启动画面
7. `vTask_Sensor` 延迟 2 秒后初始化 MPU6050, 随后进入 200 ms 采集循环
8. `vTask_Display` 延迟 5 秒后进入主界面渲染循环
9. `vTask_Menu` 和 `vTask_Bluetooth` 同步运行

3.2.3 关键技术实现

(1) SSD1306 OLED 初始化序列

SSD1306 上电后需按特定顺序发送约 20 条配置命令, 包括时钟分频、多路复用率、电荷泵使能、对比度等。以下为初始化序列的关键片段:

Listing 1: SSD1306 初始化代码片段 (`ssd1306.c`)

```

1 void SSD1306_Init(void) {
2     HAL_Delay(100);           // 上电等待

```

```

3   SSD1306_WriteCommand(0xAE);           // 关闭显示
4   SSD1306_WriteCommand(0x20);           // 设置寻址模式
5   SSD1306_WriteCommand(0x00);           // 水平寻址模式
6   SSD1306_WriteCommand(0xA1);           // 段重映射（左右镜像）
7   SSD1306_WriteCommand(0xC8);           // COM 扫描方向（上下镜像）
8   SSD1306_WriteCommand(0xA8);
9   SSD1306_WriteCommand(0x3F);           // MUX = 64（128×64 面板）
10  SSD1306_WriteCommand(0x8D);
11  SSD1306_WriteCommand(0x14);           // 启用电荷泵（3.3V 必须）
12  SSD1306_WriteCommand(0x81);
13  SSD1306_WriteCommand(0xCF);           // 对比度 = 207
14  // ... 其余约 10 条配置命令
15  SSD1306_WriteCommand(0xAF);           // 开启显示
16 }

```

（2）MPU6050 传感器 I2C 读写

MPU6050 通过 I2C1 总线与 MCU 通信，读写寄存器使用 HAL_I2C_Mem_Write/Read 函数：

Listing 2: MPU6050 I2C 读写（mpu6050.c）

```

1  // 写单个寄存器
2  static HAL_StatusTypeDef MPU6050_WriteReg(uint8_t reg, uint8_t val) {
3      return HAL_I2C_Mem_Write(&hi2c1, MPU6050_ADDR << 1, reg,
4                               I2C_MEMADD_SIZE_8BIT, &val, 1, 100);
5  }
6
7  // 读取 14 字节传感器数据（加速度+温度+陀螺仪）
8  static void MPU6050_ReadAll(MPU6050_Data *data) {
9      uint8_t buf[14];
10     HAL_I2C_Mem_Read(&hi2c1, (MPU6050_ADDR << 1) | 1,
11                     MPU6050_ACCEL_XOUT_H, I2C_MEMADD_SIZE_8BIT,
12                     buf, 14, 200);
13     data->accel_x = (int16_t)((buf[0] << 8) | buf[1]) / 16384.0f;
14     data->accel_y = (int16_t)((buf[2] << 8) | buf[3]) / 16384.0f;
15     data->accel_z = (int16_t)((buf[4] << 8) | buf[5]) / 16384.0f;
16     data->temp     = ((buf[6] << 8) | buf[7]) / 340.0f + 36.53f;
17     // ... 陀螺仪数据解析
18 }

```

（3）FreeRTOS 多任务互斥锁设计

OLED（I2C1）和 MPU6050（I2C1 共享总线）被多个任务访问。本项目使用单一互斥锁 mutexI2CHandle 保护所有 I2C 操作。关键设计决策：各外设驱动函数内部自行加

锁/解锁（如 SSD1306_SendByte、MPU6050_ReadAll），任务层不再重复加锁。此设计避免了双重加锁导致的死锁问题——该问题曾耗费大量时间排查：taskSensor 和 taskDisplay 各持锁一次后又调用内部加锁函数，造成永久阻塞。

（4）计步算法

基于 MPU6050 三轴加速度计数据，通过合成加速度、IIR 低通滤波（ $\alpha = 0.2$ ）、动态阈值（最近 5 次有效波峰均值的 52%）和峰值检测实现实时计步。最小步间隔设为 200 ms（对应最快 300 步/分钟），有效滤除了挥手、晃动手臂等非行走动作的干扰。

4 功能展示

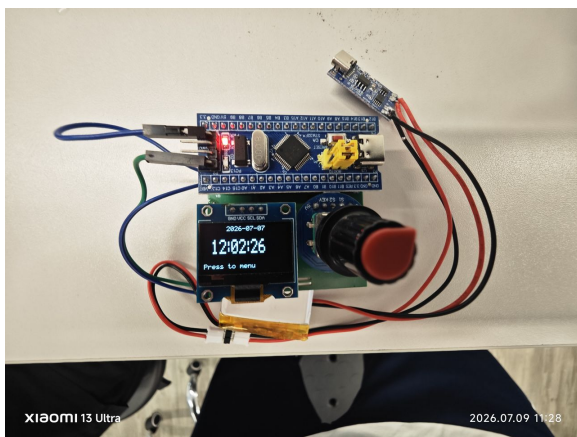
4.1 功能描述

本智能手表共实现了 11 项功能模块，按用户操作流程依次为：主界面 → MENU 主菜单 → 各子功能页面 → 设置子菜单。

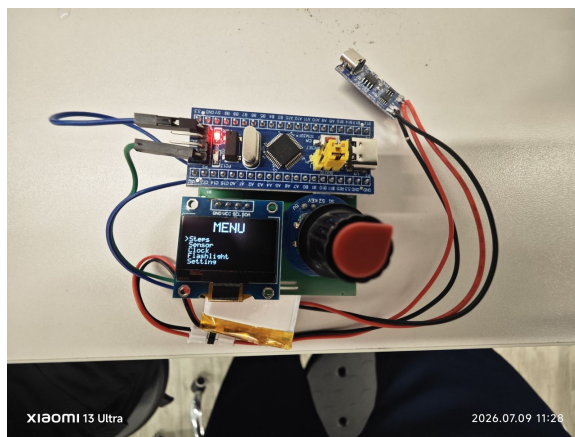
4.2 演示结果

4.2.1 主界面与菜单系统

系统启动后显示主界面，包含当前时间、日期、累计步数。按压 EC11 编码器进入 MENU 主菜单，旋转翻页选择功能子菜单。

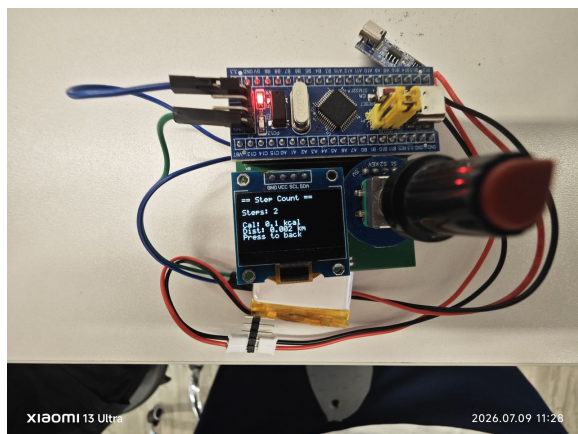


(a) 主界面——时间日期与步数

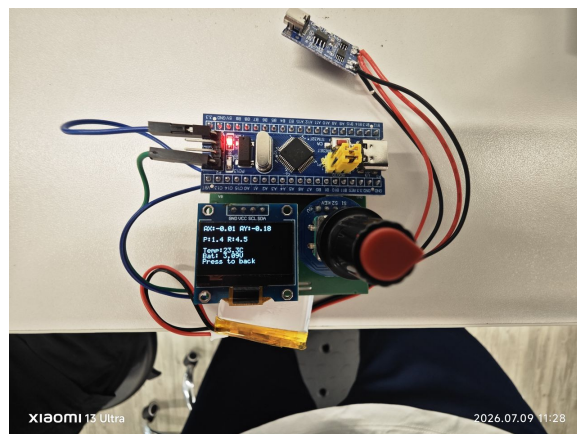


(b) MENU 主菜单

4.2.2 核心功能页面



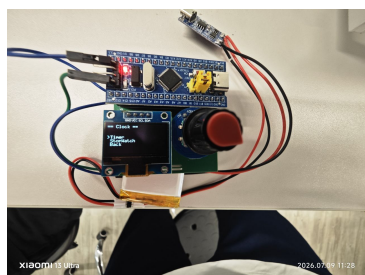
(a) STEPS——步数/卡路里/距离



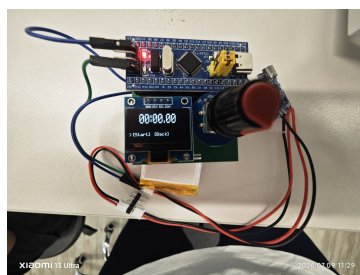
(b) SENSOR——传感器数据

4.2.3 时钟工具

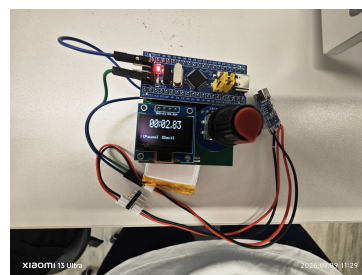
CLOCK 子菜单提供秒表和倒计时器两个实用工具。



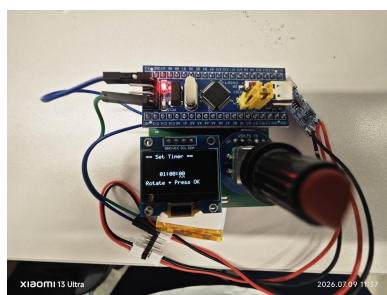
(a) Clock 子菜单



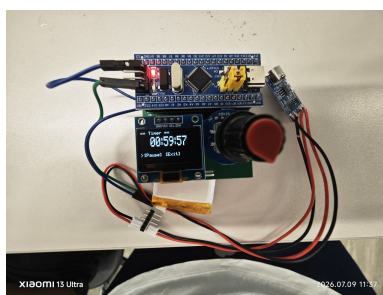
(b) 秒表-未开始



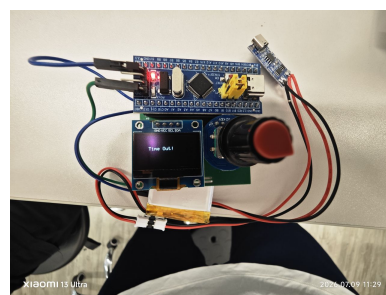
(c) 秒表-计时中



(a) 倒计时-设置时间

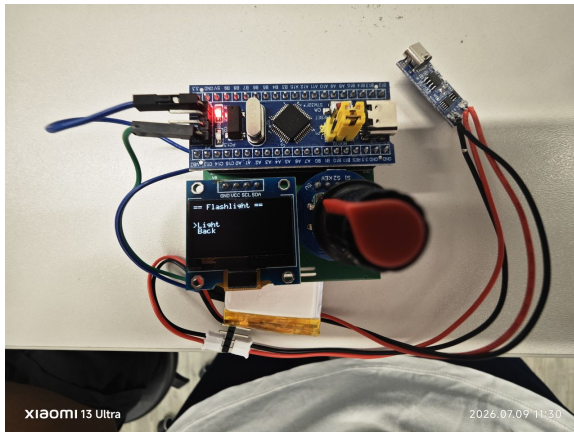


(b) 倒计时-运行中

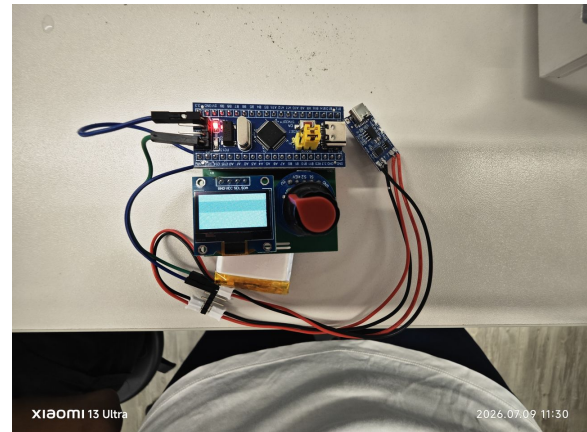


(c) 倒计时-已结束

4.2.4 手电筒功能

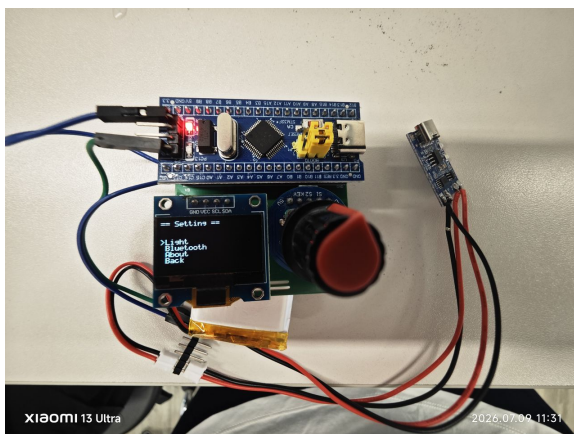


(a) 手电筒页面

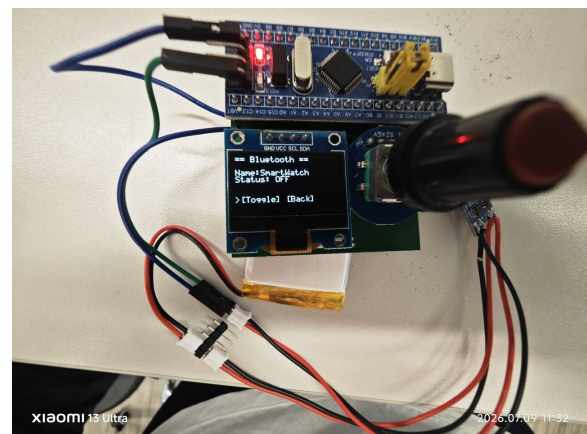


(b) 手电筒演示效果

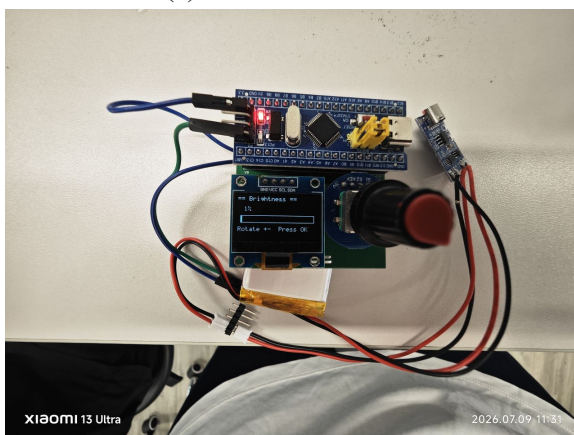
4.2.5 系统设置



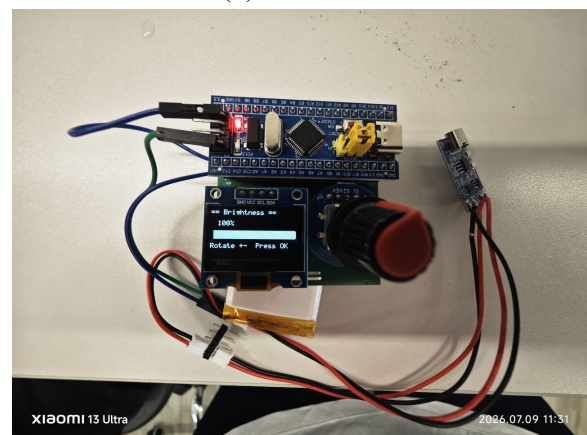
(a) SETTING 子菜单



(b) 蓝牙开关



(c) 亮度最低

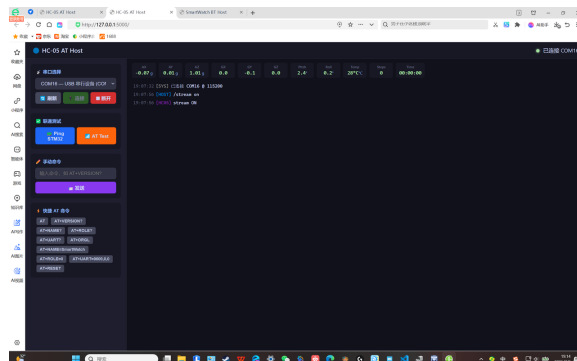


(d) 亮度最高

4.2.6 蓝牙传输与 PC 上位机



(a) 手机蓝牙接收传感器数据



(b) Flask PC 上位机面板

4.2.7 自动休眠

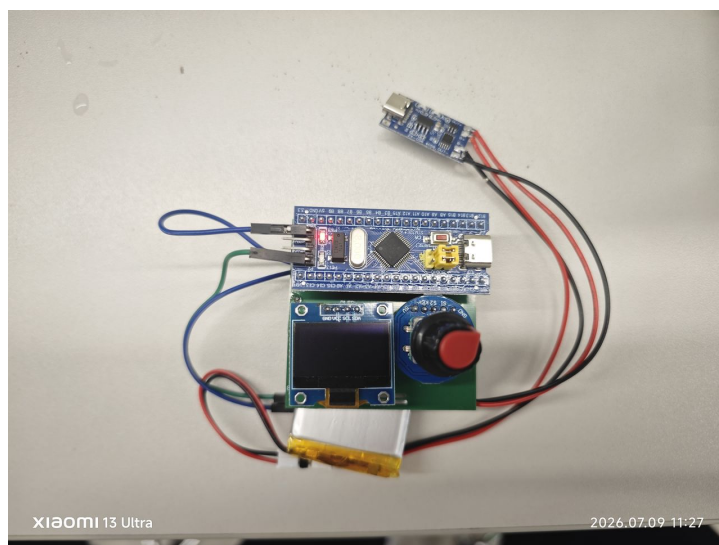


图 11 自动息屏状态（主界面 5 秒无操作触发）

4.2.8 关于页面——多字体中英文显示

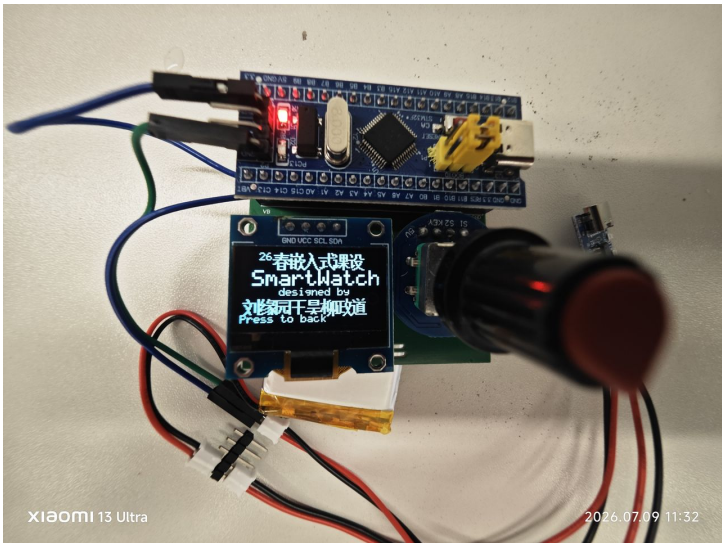


图 12 关于页面（中英文混合显示效果）

5 物料与成本

5.1 物料清单

表 4 物料清单与成本明细

序号	物料名称	型号/规格	数量	单价 (元)	用途
1	OLED 显示屏	0.96" SSD1306 I2C	1	12	显示输出
2	MPU6050 模块	GY-521	1	5	运动数据采集
3	蓝牙模块	HC-05	1	10	无线数据传输
4	旋转编码器	EC11 + OLED 一体化	1	2	菜单导航
5	稳压模块	AMS1117-3.3V	1	1.5	锂电池稳压至 3.3V
6	锂电池	3.7V 2000mAh	1	15	便携供电
7	充电模块	TP4056 Type-C	1	3	锂电池充电管理
8	MCU 最小系统	STM32F103C8T6	1	15	核心计算控制
9	调试器	ST-Link V2	1	8	程序烧录与调试
10	杜邦线	公母各 20 根	1 套	3	模块间连接
11	面包板	5×7 cm	1	3	原型搭建
				合计	77.5

5.2 项目总成本

本项目物料总成本约为 77.5 元，在嵌入式系统课程设计的预算范围内。

5.3 损耗与损坏记录

本项目开发过程中未造成任何物料损坏。所有模块在使用后均完好回收。

6 个人贡献

6.1 编码工作

本人独立完成了以下编码任务：

1. **Flask PC 上位机** (`hc05_at_host/hc05_at_host.py`)：基于 Python Flask 框架开发 Web 端实时传感器数据可视化面板，实现 COM 口连接管理、传感器数据文本流解析 (`S|ax,ay,az,...` 格式)、网页端 11 项传感器数据的实时刷新显示，并支持 `/ping` (设备在线检测)、`/oled` (OLED 重新初始化)、`/mpu` (I2C2 设备扫描)、`/stream on/off` (传感器数据流开关) 等交互式诊断命令。约 280 行 Python 代码。

6.2 硬件连接与调试

1. 与组员共同完成前期面包板电路搭建，包括 STM32 最小系统、OLED、MPU6050、HC-05 等各模块的杜邦线连接与基本功能验证
2. 独立完成锂电池供电链路的搭建与调试：TP4056 充电管理模块焊接、AMS1117-3.3V 稳压模块接线、锂电池接入与充放电测试，确保系统可脱离 USB 线独立运行

6.3 报告撰写

本人负责或参与撰写了以下项目文档：

1. 《项目计划书》：项目选题论证、开发路线规划、物料清单编制
2. 《系统设计文档》：系统架构设计、模块接口定义、FreeRTOS 任务通信方案
3. 《中期检查报告》：中期进度汇总、硬件/软件调试情况记录、存在的问题与风险分析
4. 本课程设计个人总结报告的全文撰写与 LaTeX 排版

6.4 资本投入

本人与组员共同承担了项目中各模块的采购费用（详见物料清单），并参与器件的选型与购买。

6.5 开发问题描述与解决方案

表 5 主要开发问题与解决方案

问题现象	解决方案
FreeRTOS 加入后屏幕黑屏、启动卡死	osDelay() 在调度器启动前被调用，导致硬件错误。初始化阶段改用 HAL_Delay(), 任务内部保留 osDelay()
调度器启动后所有任务无法延时（osDelay 不返回）	FreeRTOS SysTick CTRL 未使能（实际值为 0x0010 而非预期 0x0007），滴答中断不触发。在 MX_FREERTOS_Init 中手动配置 SysTick 寄存器解决
MPU6050 初始化时系统卡死	I2C 通信超时参数为 HAL_MAX_DELAY（无限等待），设备未应答时永久阻塞。改为 200 ms 有限超时
多任务并发时 OLED 渲染卡死（死锁）	任务层与驱动层各锁一次互斥锁。去除任务层外层加锁，仅在内层 I2C 操作函数中保护
蓝牙输出数据全为 0x00	HC-05 波特率与 STM32 USART 不匹配。排查后统一为模块实际波特率
浮点数值在 OLED 上显示为空	newlib-nano 标准库不支持 %f 格式化。实现 fmt_fixed() 函数将浮点数转为整数形式输出
锂电池单独供电不工作	初次使用电池保护板锁死。先插 Type-C 充电 15 分钟激活保护板后正常

7 项目成果与反思

7.1 成果展示

本项目成功实现了一款基于 STM32F103C8T6 的多功能智能手表原型，主要成果包括：

- 完整的传感器 → MCU → OLED 本地显示 → 蓝牙无线传输 → PC 上位机全链路打通
- FreeRTOS 5 任务架构稳定运行，各功能模块可长时间连续工作

- 超出课程文档要求的创新功能：秒表/倒计时、手电筒、亮度进度条、中英文混合显示、自动休眠、蓝牙开关
- 锂电池独立供电链路，可脱离 USB 线手持演示
- 关键底层技术突破：SysTick 初始化失败——一个官方文档未覆盖的兼容性问题——被成功排查并修复

7.2 项目评估

7.2.1 成功之处

1. 渐进式开发有效降低了调试难度：从裸机 OLED 到 MPU6050 到蓝牙到 FreeRTOS，每个阶段只新增一个模块，确保始终有一个可验证的基线
2. 底层排查能力的提升：在 SysTick 未使能这一问题上，从最初的怀疑堆栈溢出、互斥锁死锁，到最终通过 TIM2 硬件定时器中断直接读取 SysTick CTRL 寄存器定位根因，体现了从现象到本质的排查思路
3. 架构设计的简洁性：5 任务 + 1 互斥锁的方案避免了队列带来的额外堆开销，在 20 KB 的极限 RAM 约束下实现了稳定调度
4. 代码的可维护性：模块化驱动设计使得每个外设可以独立测试和替换
5. 功能完整性：在老师要求基础上自主拓展了 10 项创新功能，展示了软件架构的可扩展性

7.2.2 不足之处

1. **RAM 使用率偏高（93%）**：当前 20 KB RAM 中仅余约 1.4 KB。后续可进一步精简任务栈和 USB CDC 缓冲区，或评估外扩 SRAM 的可行性
2. **计步精度待量化**：当前缺少与商用计步器在同一场景下的对比测试数据，精度仅凭经验估计
3. **无专用手机 APP**：手机端只能通过通用蓝牙串口 APP 以文本格式查看数据，缺少图表化展示
4. **系统功耗未精确测量**：各模块的独立功耗数据缺失，低功耗策略的优化缺乏量化依据

5. 中文字库维护不便：当前中文按需生成字模的方法虽节省了 Flash，但新增汉字需要重新运行 Python 脚本生成字模、重新编译固件

7.3 个人反思

本课程设计最大的收获来自 FreeRTOS 迁移阶段。加入操作系统后系统黑屏、启动卡死、任务无法延时等问题消耗了近 10 小时的调试时间。最终发现根因是 SysTick 控制寄存器——一个 Cortex-M3 核内、位于 0xE000E010 地址的寄存器——缺乏一个底层的了解——直接读写芯片寄存器来验证操作系统层的假设，这种从最底层向上排查的思路远比依赖 printf 或调试器更加高效。

另一点体会是：在资源极度受限的嵌入式平台（20 KB RAM，64 KB Flash）上，每一个字节都需要精打细算。USB CDC 缓冲区从 1024 字节缩减到 256 字节，任务栈从默认的 1024 字缩减到 384 字，每一项优化都需要在功能和稳定性之间做权衡。这种在约束条件下做工程决策的能力，是嵌入式系统设计与 PC 端软件开发最大的不同。

在与组员的协作中，通过定期的代码同步和模块接口约定，确保了各自独立开发的驱动代码能够顺利集成到统一的 FreeRTOS 架构中，这也锻炼了团队协作和项目管理能力。

8 附录

8.1 设计图纸

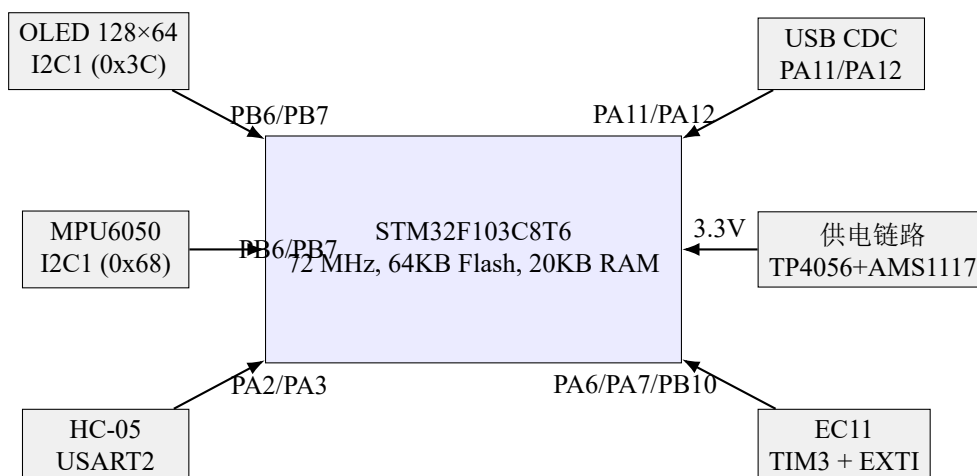


图 13 系统硬件架构图

8.2 完整代码清单

表 6 核心代码文件清单

文件	说明
Core/Src/main.c	主程序入口，外设初始化
Core/Src/freertos.c	FreeRTOS 5 任务实现，SysTick 修复
Core/Src/ssd1306.c	/ OLED 显示驱动（SSD1306/SH1106）
Core/Inc/ssd1306.h	
Core/Src/mpu6050.c	/ MPU6050 六轴传感器驱动
Core/Inc/mpu6050.h	
Core/Src/bluetooth.c	/ 蓝牙帧协议驱动
Core/Inc/bluetooth.h	
Core/Src/encoder.c	/ EC11 编码器状态机
Core/Inc/encoder.h	
Core/Src/stopwatch.c	/ 秒表功能
Core/Inc/stopwatch.h	
Core/Src/timer.c	/ 倒计时器功能
Core/Inc/timer.h	
Core/Inc/menu.h	多级菜单数据结构定义

8.3 参考文献

1. STM32F103C8T6 Datasheet, STMicroelectronics, 2023
2. SSD1306 Advance Information, Solomon Systech, 2008
3. MPU-6050 Datasheet, InvenSense, 2013
4. Mastering the FreeRTOS Real Time Kernel, Richard Barry, 2016
5. STM32CubeMX User Manual, STMicroelectronics, 2024
6. ARM Cortex-M3 Technical Reference Manual, ARM Ltd., 2010
7. 周贤中. 嵌入式系统设计课程讲义, 广东工业大学, 2026